

TESP Inteligência Artificial

Programação de Computadores II

Python + MySQL

Persistência de dados

- Apesar de já termos abordado persistência de dados com recurso a ficheiros, vamos abordar agora, persistência de dados com recurso a base de dados MySQL.

Requisitos

- Primeiro passo, abrir a interface de linha de comandos e digitar o comando:

```
pip install mysql-connector
```

- Aguarde o download e o driver estará pronto para utilizarmos. Agora vamos confirmar se ele foi instalado corretamente:

```
import mysql.connector
```

Dados de ligação com a BD

- ❑ Caso o comando acima tenha sido executado sem nenhum erro, estamos prontos para continuar!
- ❑ O primeiro passo agora é criar a ligação com a base de dados

```
import mysql.connector

db = mysql.connector.connect(
    host="localhost",
    user="usuario",
    passwd="senha"
)

print(db)
# <mysql.connector.connection.MySQLConnection object at 0x7f66da2a7550>
```

Criar uma base de dados pelo Python

- Agora vamos criar uma base de dados chamada banco, para isso vamos utilizar o comando SQL "CREATE DATABASE". Precisamos de definir um cursor que nos permite executar comandos SQL

```
import mysql.connector

db = mysql.connector.connect(
    host="localhost",
    user="usuario",
    passwd="senha"
)

cursor = db.cursor()
cursor.execute("CREATE DATABASE banco")
```

Mostrar bases de dados

- Para verificar se a base de dados que criamos realmente está presente, utilizaremos o comando "SHOW DATABASES"

```
import mysql.connector

db = mysql.connector.connect(
    host="localhost",
    user="usuario",
    passwd="senha"
)

cursor = db.cursor()

cursor.execute("SHOW DATABASES")

for c in cursor:
    print(c)
```

Abrir a base dados na ligação

- Podemos também aceder a base de dados no momento da ligação:

```
import mysql.connector

db = mysql.connector.connect(
    host="localhost",
    user="usuario",
    passwd="senha",
    database="banco"
)
```

Criar uma tabela

□ Através do comando “CREATE TABLE”

```
import mysql.connector

db = mysql.connector.connect(
    host="localhost",
    user="usuario",
    passwd="senha",
    database="banco"
)

cursor = db.cursor()
cursor.execute("CREATE TABLE filmes (nome VARCHAR(255), gênero VARCHAR(255))")

cursor.execute("SHOW TABLES")
for c in cursor:
    print(c)
# ('filmes',)
```


Chave Primária

- Ao criarmos uma tabela, é importante que tenhamos uma coluna que possua uma chave única para cada registro, chave primária, para isso vamos utilizar o comando "INT AUTO_INCREMENT PRIMARY KEY" que irá inserir um número único para cada registro, começando por 1 e incrementando para cada registro que vier.

```
cursor.execute("CREATE TABLE filmes (id INT AUTO_INCREMENT PRIMARY KEY, nome VARCHAR(255), endereco VARCHAR(255))")
```

- Veja que ocorre um problema, pois já possuímos a tabela, para contornarmos essa situação, vamos utilizar o comando "ALTER TABLE", que irá alterar a tabela com essa nova coluna.

```
cursor.execute("ALTER TABLE filmes ADD COLUMN id INT AUTO_INCREMENT PRIMARY KEY")
```

Inserir dados

- De nada adianta termos uma tabela se não for para preenchermos com valores para uso futuro. Para essa tarefa vamos utilizar o comando "INSERT INTO".

```
import mysql.connector

db = mysql.connector.connect(
    host="localhost",
    user="usuario",
    passwd="senha",
    database="banco"
)

cursor = db.cursor()

comando_sql = "INSERT INTO filmes (nome, gênero) VALUES (%s, %s)"
dados = ("2001: A Space Odyssey", "Science Fiction")
cursor.execute(comando_sql, dados)

db.commit()

print(cursor.rowcount, "registro inserido") # 1 registro inserido
```

Inserir múltiplas linhas

```
import mysql.connector

db = mysql.connector.connect(
    host="localhost",
    user="usuario",
    passwd="senha",
    database="banco"
)

cursor = db.cursor()
comando_sql = "INSERT INTO filmes (nome, gênero) VALUES (%s, %s)"
valores = [
    ("The Lord of the Rings: The Fellowship of the Ring", "Fantasy"),
    ("Star Wars: Episode V - The Empire Strikes Back", "Space Opera"),
    ("Andrei Rublev", "Drama")
]

cursor.executemany(comando_sql, valores)

db.commit()

print(cursor.rowcount, "filmes foram inseridos") # 3 filmes foram inseridos
```

Obter ID do último valor inserido

- Podemos obter o id da linha que acabamos de inserir perguntando ao objeto cursor:

```
print("1 registro inserido, ID:", cursor.lastrowid) # 1 registro inserido, ID: 2
```

Selecionar dados da tabela

```
import mysql.connector

db = mysql.connector.connect(
    host="localhost",
    user="usuario",
    passwd="senha",
    database="banco"
)

cursor = db.cursor()

cursor.execute("SELECT * FROM filmes")

resultados = cursor.fetchall()

for resultado in resultados:
    print(resultado)
# ('2001: A Space Odyssey', 'Science Fiction', 1)
# ('The Lord of the Rings: The Fellowship of the Ring', 'Fantasy', 2)
# ('Star Wars: Episode V - The Empire Strikes Back', 'Space Opera', 3)
# ('Andrei Rublev', 'Drama', 4)
```

Selecionar dados da tabela

- ❑ O * em SELECT especifica que queremos todos os registros da tabela, usamos o método fetchall() que retorna todas as linhas do nosso último comando executado.
- ❑ Também podemos selecionar apenas determinadas colunas da tabela, por exemplo:

```
cursor.execute("SELECT nome, gênero FROM usuario")  
cursor.execute("SELECT nome FROM usuario")
```

Selecionar dados da tabela

- ❑ O método `fetchone()` traz apenas uma linha da tabela, ele devolve apenas a primeira do resultado:

```
import mysql.connector

db = mysql.connector.connect(
    host="localhost",
    user="usuario",
    passwd="senha",
    database="banco"
)

cursor = db.cursor()

cursor.execute("SELECT nome FROM filmes")

resultado = cursor.fetchone()
print(resultado) # ('2001: A Space Odyssey',)
```

Eliminar dados

- Podemos também deletar registros da nossa tabela, para isso vamos usar o comando "DELETE FROM".

```
import mysql.connector

db = mysql.connector.connect(
    host="localhost",
    user="usuario",
    passwd="senha",
    database="banco"
)

cursor = db.cursor()

sql = "DELETE FROM filmes WHERE nome = 'Andrei Rublev'"
cursor.execute(sql)
db.commit()

print(cursor.rowcount, "registro excluído") # 1 registro excluído
```


Eliminar tabela

- Podemos facilmente deletar uma tabela com o comando "DROP TABLE":

```
import mysql.connector

db = mysql.connector.connect(
    host="localhost",
    user="usuario",
    passwd="senha",
    database="banco"
)

cursor = db.cursor()

sql = "DROP TABLE filme"

cursor.execute(sql)
```

Eliminar tabela

- Se a tabela que deseja excluir já foi excluída ou, por qualquer outro motivo, não existe, pode usar a palavra-chave **IF EXISTS** para evitar um erro.

```
import mysql.connector

db = mysql.connector.connect(
    host="localhost",
    user="usuario",
    passwd="senha",
    database="banco"
)

cursor = db.cursor()

sql = "DROP TABLE IF EXISTS filme"

cursor.execute(sql)
```

Atualizar dados

- Podemos utilizar o comando "UPDATE", vamos alterar o gênero do filme 2001: A Space Odyssey para Ciência.

```
import mysql.connector

db = mysql.connector.connect(
    host="localhost",
    user="usuario",
    passwd="senha",
    database="banco"
)

cursor = db.cursor()

comando_sql = "UPDATE filmes SET gênero = 'Ciência' WHERE nome = '2001: A Space Odyssey'"

cursor.execute(comando_sql)

db.commit()

print(cursor.rowcount, "registro afetado") # 1 registro afetado
```